

Subprograms

Revise 360 lesson plan — 60 minutes

Unit	Beyond OCR J277: a Python pathway supporting 2.1, 2.2 and 2.3
Lesson	Subprograms (lesson 9 of the unit)
Lesson ID	pr-109
Experience ID	pr-109
Plan ID	PLAN-PR-L09
Content version	5eda7c15ccbe
Website	https://revise360.co.uk/experience.html?id=pr-109

Big question, outcomes and success criteria

Big question — When should a subprogram return a value rather than print one?

Prerequisites

The lesson before this one is pr-108. Pupils should already have finished it: the starter below retrieves from it.

Earlier in the unit: pr-101, pr-102, pr-103, pr-104, pr-105, pr-106, pr-107.

Assessable outcomes

- pr-109-o1** — know how to define a subprogram with def and how to call it
- pr-109-o2** — understand how a parameter carries a value into a subprogram and how return carries one back out
- pr-109-o3** — know how to write a subprogram that uses a variable of its own, and one that changes a shared variable

Success criteria the pupil can check against

Taken from the briefs the independent activities themselves show, so these are the words the pupil sees. Their work has met the outcome when it does these things.

- Count how many lines of output two calls give you. **pr-109-s1-a1**
- Do not change the procedure itself. **pr-109-s1-a3**
- Display exactly: --- **pr-109-s1-a3**
- Three lines of output. **pr-109-s1-a3**
- Replace ____ with the line that calls cheer. **pr-109-s1-a4**
- Display exactly: Well done **pr-109-s1-a4**
- One line of output. **pr-109-s1-a4**
- The user types in one word. **pr-109-s1-a5**

Vocabulary, resources and preparation

Key vocabulary — Subprogram, Parameter and argument, return, Local and global variable

Resources

SLIDES	presentations/PR_L09_Subprograms_Lesson.pptx
WORKSHEET	worksheets/PR_L09_Subprograms_Worksheet.docx
ONLINE EXPERIENCE	experience.html?id=pr-109

Before the lesson

1. Sign in on the **teacher dashboard** (`teacher.html`) and open **Class logins**. Enter the class or group and the usernames, one per line, then print the login cards. A pupil cannot sign in without one, so do this before the lesson rather than in it.
2. Print the worksheet, one per pupil.
3. Open the slides and the experience once yourself, on the machine you will be teaching from, so the Python runtime and the 360 image are already cached.
4. Check headsets or devices are charged and on the school network.

How pupils get in

1. Go to `topics.html` on `revise360.co.uk`.
2. Sign in with the **username and PIN from the login card**. The class is already set against the card.
3. Choose the topic, then this lesson's card.

Worth knowing before you promise anything. In this build `APP_CONFIG.backendUrl` is empty, so logins, login cards and progress are stored on the device the pupil used. The teacher dashboard says so too. A pupil who moves to a different machine starts again, so keep a class on the same devices within a unit, and treat the paper worksheet as the record that travels.

Teaching sequence

Sixty minutes including transitions. The online block is the one to move if your lesson is shorter: see the adaptation at the end.

MIN	PHASE	TEACHER	PUPILS	EVIDENCE TO CHECK
0-5	Retrieve	Starter on the board or the worksheet, from the last lesson and this lesson's prerequisites.	Answer the starter on the worksheet.	The starter box on <code>PR_L09_Subprograms_Worksheet.docx</code> is filled in.
5-8	Review the starter	Take answers; correct the one most got wrong before moving on.	Correct their own starter.	A correction written, not just a tick.
8-16	Explain and model	The new idea, then the worked example, typed live rather than shown finished.	Watch the example being built, and say what each line does.	They can name the line that does the job, not just copy it.
16-19	Set up and settle	Sign-in, launch, and what to record on paper before anyone types.	Sign in, open the lesson, read the task before typing.	Everyone is at station 1 activity 1, not typing yet.
19-48	Online: work the stations in order	Pupils work; you circulate and read the editor over shoulders.	Work the stations in order, running and checking as they go.	The station badges fill in; the editor shows their own program, not the starter.
48-53	Check understanding	The hinge question, hands down, then the reteach if the class splits.	Answer the hinge question, hands down.	The split across the options, before any reteach.
53-58	Apply or extend	The challenge activity, or the exam practice on the worksheet.	The Challenge activity, or the exam practice.	worksheet, Exam practice a) to c)
58-60	Exit and set up the next lesson	The exit question and what to finish before next time.	Write the exit answer.	One sentence that answers the big question.

The explanation and the worked example

This is what the walls of the experience say, station by station. Teach the first two or three from the front before anyone puts a headset on; the rest the pupils read for themselves.

1. Writing a procedure

- `def` names a subprogram, and the indented block below it is what that subprogram does.
- Nothing happens until the name is called, with brackets after it.
- Common mistake: defining a procedure and never calling it, so none of it ever runs.

Challenge on this wall: What is the difference between defining a procedure and calling it?

2. Parameters

- A parameter in the brackets lets whoever calls the subprogram pass a value in.
- The value handed over is the argument, and inside it is used like any other variable.
- Common mistake: giving the arguments in a different order from the parameters.

Challenge on this wall: Why is a procedure with a parameter more useful than one without?

3. Returning a value

- `return` hands a value back to whatever called the subprogram.
- The call can then be stored in a variable, printed, or used inside a bigger sum.
- Common mistake: printing inside when the task says `return`, so nothing is handed back.

Challenge on this wall: Why would a task insist a subprogram returns a value rather than prints it?

4. Functions and procedures

- A function returns a value; a procedure just does a job and returns nothing.
- Both are subprograms, and in Python both are written with `def`.
- Common mistake: calling something a function when all it does is `print`.

Challenge on this wall: Decide whether each subprogram in your program is a function or a procedure.

5. Local and global

- A variable made inside a subprogram is local: it exists only while that subprogram runs.
- A global variable belongs to the main program, and `global` lets a subprogram change it.
- Common mistake: expecting a local variable's value to survive after the subprogram ends.

Challenge on this wall: Why can two subprograms both use a variable called `total` without clashing?

6. Putting it together

- A subprogram may call another one, so a big job splits into small named steps.
- Calling a function inside a loop lets one piece of code answer the same question many times.
- Common mistake: repeating the same lines instead of writing one subprogram and calling it.

Challenge on this wall: Which parts of a long program are worth turning into subprograms, and which must return a value?

The example to model

A subprogram is a named piece of a program you can use again. `def` gives the piece a name, and the indented lines under it belong to that name. Writing the name with brackets after it runs those lines, and that is called calling it. A subprogram that only displays things, like this one, is called a procedure.

```
def welcome():
    print("Doors open")
    print("Mind the step")
welcome()
welcome()
```

It displays **Doors open / Mind the step / Doors open / Mind the step**

Decisions to model out loud, not just the finished answer: say why you chose that name, why that line comes before the next one, and what you would try first if it did not work. The example deliberately uses different data from the task, so a pupil cannot copy it and be finished.

Retrieval and the check for understanding

Starter — A program defines a subprogram called banner with one print line inside it, and then stops. Nothing is printed when it runs. Explain why, and write down the line that has been left out.

Answer space: 3 lines on the worksheet.

The answer is in the teacher answers for this unit, not on this plan: a plan left on a desk should not give it away.

Hinge question

Use this one, hands down, before anyone moves on to independent work. It is a real activity from the experience (pr-109-s4-c2), so the wording a pupil sees matches the wording you say.

What will this program display?

```
def total(a, b):  
    print(a + b)  
answer = total(2, 3)  
print(answer)
```

Correct: 5 None

What a pupil might choose instead

- 5 5
- None 5

Count the hands on each. If more than a few of the class are on one wrong option, that option is the misconception to teach against before anyone goes on: go back to the station this question sits on and work through its example again, aloud, before the independent work.

The course's own response to a wrong answer here: The 5 on screen came from the print inside total. Nothing was handed back, so answer holds None. This is the mistake to watch for all lesson.

One response, not four. one response covers every wrong option; it does not say what to revisit for each one, so decide your own reteach for each option before the lesson - the list above is the start of it.

Misconceptions this lesson addresses

- defining a procedure and never calling it, so none of it ever runs. [wall panel](#)
- Everything is spelled correctly. Ask instead what has been made but never used. [pr-109-s1-a6, a Fix it activity](#)
- giving the arguments in a different order from the parameters. [wall panel](#)
- The value arrives under one name and one name only. Which name does the def line give it?
[pr-109-s2-a6, a Fix it activity](#)
- printing inside when the task says return, so nothing is handed back. [wall panel](#)
- The sum is right. Ask what the function does with the answer once it has worked it out.
[pr-109-s3-a6, a Fix it activity](#)
- calling something a function when all it does is print. [wall panel](#)
- Try a small number and a large one. Only one of them comes out right, so look at the two paths separately.
[pr-109-s4-a6, a Fix it activity](#)

Guided to independent practice

STAGE	ACTIVITIES	WHAT IT IS
-------	------------	------------

Recognise	12	The answer is on the screen and the pupil picks it: multiple choice, choosing several, matching a pair.
-----------	----	---

STAGE	ACTIVITIES	WHAT IT IS
Construct	11	Nothing is offered. The pupil classifies, orders, fills in a table or works a number out, and types or places the answer.
Produce	14	The pupil writes it from the brief and nothing else.
Challenge	6	The hardest activity on each station. It is not optional and it is not extra: it is the last step of the station.

Scaffolds already in the product, and how they fade

- **Hint** is a ladder, not a door: the first rung names the idea, the next shows the shape of the code, the next how it starts, the last animates the technique on different data. A pupil asks for each rung. None of them is the answer.
- **Run** is free and unmarked; **Check my answer** marks. A pupil may run as often as they like, which is what makes trial and error safe.
- **I need help** says to ask you, and says plainly that the question still has to be completed. It does not notify anyone and it does not unlock anything.
- The worked example appears on every early activity and on none of the late ones. That fade is deliberate; it is not a missing example.

Accessibility

- **Read aloud** on every task window reads the task and its brief, with the prescribed values read as values rather than as punctuation.
- Text is 16px or larger throughout, and a question window scrolls rather than shrinking its type.
- Prescribed data is boxed and in a monospace face as well as coloured, so it still reads as code in black and white, in a high-contrast mode, and to a pupil who does not separate green from amber.
- **Lower literacy and EAL:** pre-teach the vocabulary above; let the pupil say the key fact before writing it; accept the key fact in two or three words.
- **SEND:** the sequence is compulsory but the pace is not. A pupil may finish a station next lesson; the progress saves after every answer and reopens at their earliest unfinished question.
- A headset is never required. Every experience works on a laptop or a tablet with a mouse or a finger.

Delivering the online experience

Before anyone enters

- Worksheet out, pen in hand. The rule is write first, then answer at the badge: the first answer is the one that counts.
- Say where the stations are. They are numbered around the room: 1 and 2 to the right, 3 and 4 behind, 5 and 6 to the left, and the final challenge on the floor.
- Say that a badge marked **Not yet** is not a fault. It opens when the station they are on is finished.

The stations, and what to record at each

#	STATION	ACTIVITIES	RECORD ON PAPER
1	Writing a procedure	7	Writing a job once and then calling it as often as you like is known as...
2	Parameters	7	The names in the brackets of the definition are the parameters, and the values handed over at the call are the...
3	Returning a value	7	return hands the value back and ends the subprogram on the spot, so a line written after it...
4	Functions and procedures	7	The one thing that makes a subprogram a function rather than a procedure is that it...
5	Local and global	7	Keeping a variable local stops one part of a program quietly breaking...
6	Putting it together	7	Breaking a problem down into small named parts has a name you met in topic 2.1:...

#	STATION	ACTIVITIES	RECORD ON PAPER
final	Final challenge: function or procedure?	1	the answer to the station's challenge

Where to pause

Stop the class after station 2 and take answers to its challenge out loud. That is the point where a pupil who has misread the technique can still be put right cheaply.

Afterwards

Headsets off and down. Ask three pupils for the key fact from a station of your choosing, in their own words and not read off the sheet. Then the exit question.

Python: the order is not negotiable. Each activity teaches what the next one needs, so a pupil works through them in order and completes every one. A shorter lesson, homework or a slow worker does not authorise skipping: the pupil stops where they are and resumes at their earliest unfinished question next time, because progress saves after every answer.

A paper activity, a help request or a tick on this plan does not mark an online question complete, and there is no teacher bypass. If a pupil is stuck after the hint, that is the moment to teach them, not to move them on.

Assessment, evidence and unfinished work

ID	QUESTION	MARKS	WHERE
pr-109-x1	A function called <code>double</code> holds one line, <code>print(n * 2)</code> . The main program writes <code>answer = double(7)</code> and then prints <code>answer</code> , which shows <code>None</code> . Explain why <code>None</code> is printed, and write the line inside <code>double</code> corrected.	3	worksheet, Exam practice a); deck, Exam practice slide
pr-109-x2	Write a function that takes two whole numbers and returns the larger of them, and a main program that reads two numbers and prints the value the function hands back.	5	worksheet, Exam practice b); deck, Exam practice slide
pr-109-x3	Write a procedure that takes a word and a whole number and prints them on one line with a single space between them, then call it for three different pairs of values.	4	worksheet, Exam practice c); deck, Exam practice slide

Expected answers

Model answers and marking guidance are teacher-only and are not printed on this plan. They are in the unit's teacher guide pack, and the per-activity marking for this lesson is in `answers/records/pr-109.json`, which is not published with the website.

Every programming activity in this lesson is marked by running the pupil's program against its test cases, so the mark is the program's behaviour and not its appearance. The model solutions are in `answers/codebank/pr-109.json`.

Common errors

The wrong options the activities offer are the errors this lesson expects. The commonest to watch for:

- **What will this program display?** — ready start; start ready
- **What will this program display?** — 6 6; 5 6
- **What will this program display?** — 7 7; 9 9
- **What will this program display?** — 5 5; None 5
- **What will this program display?** — limit; None

Unfinished work

Record where each pupil stopped, not a tick. They resume at their earliest unfinished question, and a station left half-done is the thing to open next lesson. Do not mark the lesson complete for a pupil who did not finish it: the score screen already shows what was and was not done, and that is the honest record.

Exit, next lesson and homework

Exit check

One of these, written, not a show of hands:

- Defining a subprogram and calling it
- Passing values in with parameters and back with return
- Telling a local variable from a global one

The confidence columns on the worksheet are a supplement to this, not a substitute: a pupil who rates themselves confident and cannot answer the exit question has told you nothing.

Retrieval link to next lesson

Next is pr-110, File handling. Use this lesson's big question as next lesson's starter, so the first thing they do is retrieve it.

Homework

Finish any station left open, on the website, from the question they stopped at. If the class has no devices at home, set the exam practice on the worksheet instead and mark it next lesson.

Shorter lessons, longer lessons and no internet

If this lesson takes more than an hour

It will. There are 43 activities across 6 stations, and a beginner does not finish 43 of them in twenty-nine minutes. Plan it as two or three sessions of an hour: stations 1 and 2 in the first, 3 and 4 in the second, 5, 6 and the final challenge in the third. Each session keeps the same starter-explain-model opening; the online block resumes where the class got to. Honest total: two to three hours.

A 40 to 45 minute lesson

Keep the starter, the explanation and the model at full length: cutting those is what makes the online work fail. Take the time out of the online block and the apply phase. Pupils stop where they are; their progress saves after every answer and reopens at their earliest unfinished question, so the next lesson starts there.

A shorter lesson still does not authorise skipping a question. The save-and-resume point is wherever the pupil got to; there is no way to mark a question complete without completing it, and no teacher bypass.

If the internet or the devices are unavailable

Teach from the slides and the worksheet. The slides carry the same teaching as the station walls, and the worksheet carries the key facts, the challenges and the exam practice. Set the online stations when the devices are back.

There is no offline version of the experience, and this plan does not contain one. Do not hand out a printed walkthrough of the stations as a substitute: the experiences are used on the website.

Revise 360 — a product of Revise 360 Ltd, created by Olly Pettitt. An independent resource: not written, endorsed or approved by OCR or any exam board. Classroom use and personal study only.